

1st Term Project for the Course

Neural Networks and Deep Learning

Aimilios Mavrothlassitis (AEM 10983)
Department of Computer Science
Aristotle University of Thessaloniki (AUTH)
Thessaloniki, Greece
amavroth@ece.auth.gr

All source code used in this project, including the implementations of nearest neighbours, centroid classifiers, and the fully connected neural network, is available in my GitHub repository: <https://github.com/EmiliosBlacksea/Fully-Connected-Neural-Network-from-scratch>.

Abstract—This project is about building a Neural Network and understand how its fundamental parameters affect its performance. I chose this class because of my strong interest in Artificial Intelligence, despite having little to no knowledge of the field. This report explains the purpose of the project, the methods used, and discusses how successfully the objectives were achieved.

I. INTRODUCTION

Neural Networks have a fundamental role in ML and AI, they are inspired by the way human brains work and learn via training in large amounts of Data.

This projects goal is the classification of CIFAR-10 dataset. The project has 2 main parts: 1 - 3 nearest neighbors and the compare to Centroids methods, simple Neural Network from scratch.

The methods will serve as a baseline to compare with the neural network's performance.

II. IMPLEMENTATION OF KNN AND CENTROIDS METHODS

A. Managing Data

To use the images from Cifar-10 come in an unusable format so here is the way to turn each image into vector.

After downloading the dataset, I had data batches 1 to 5 along with a test batch in an unusable format, so I defined the given unpickle function:

Listing 1. Function Unpickle

```
import pickle

def Unpickle(file):
    with open(file, 'rb') as fo:
        data_dict = pickle.load(fo, encoding='bytes')
    return data_dict
```

data_dict is a dictionary containing the data for all images along with their labels. The Load data function makes the data_batch and labels_batch arrays which contain the data and labels for all images

Listing 2. Fuction Load data

```
from functions.unpickle import Unpickle
import numpy as np

#RETURNS labels_batch_1->5, data_batch_1->5,
#labels_training, data_training
def Load_data():
    #Get labels for "training" images
    databatch1 = r"path to the databatch file"
    labels_batch_1 = np.array(Unpickle(databatch1)[b'labels']).astype(int)
    #Get data for "training" images
    data_batch_1 = np.array(Unpickle(databatch1)[b'data']).astype(int)
    return (
        labels_batch_1, labels_batch_2,
        labels_batch_3, labels_batch_4,
        labels_batch_5,
        data_batch_1, data_batch_2, data_batch_3,
        data_batch_4, data_batch_5,
        labels_testing, data_testing
    )
```

In my original code I have done the same for all the batches. From now on any function that requires the data will use

```
(
    labels_batch_1, labels_batch_2, labels_batch_3,
    labels_batch_4, labels_batch_5,
    data_batch_1, data_batch_2, data_batch_3,
    data_batch_4, data_batch_5,
    labels_train,
    data_train
) = Load_data()
```

in the beginning.

B. Nearest Neighbors

The first algorithm that I recreated was the *k-Nearest Neighbors* (k-NN) classifier. The way the algorithm works is by vectorizing all training images and comparing to all of them a testing image. The label of the closest vector to our test image wins.

Since we were guided to test 1 and 3 nearest neighbors I implemented the function

Listing 3. Find_3_nn

```
def Find_3_nn(test_data, data_batch, labels_batch):
    distances_labels = [[inf, -1], [inf, -1], [inf, -1]]
    for i in range(len(data_batch)):
        cur = distances_labels[2][0]
```

```

pot = np.sum((test_data - data_batch[i]) **
2) # Withought sqrt I am comparing
distances ^ 2 to distances ^ 2 so
closest 3 n withh still be the same
if pot < cur:
    distances_labels[2][0] = pot
    distances_labels[2][1] = labels_batch[i]
    distances_labels.sort(key=lambda x: x
[0])
return distances_labels

```

Let distance_labels be a 2×3 array:

$$\text{distance_labels} = \begin{bmatrix} d_{(1)} & d_{(2)} & d_{(3)} \\ \ell_{(1)} & \ell_{(2)} & \ell_{(3)} \end{bmatrix},$$

where the first row contains the three smallest distances from the test image to the vectors in data_batch, and the second row contains the corresponding class labels.

To evaluate k-NN, I compute 3-NN for all batches and concatenate the results into the vector all_nn, which aggregates distances and labels from every batch. Sorting all_nn yields the actual three nearest neighbors in the first three columns.

Prediction. For 1-NN, the predicted label is the label of the single closest neighbor, and the prediction is correct if it matches the ground-truth label. For 3-NN, the predicted label is obtained by majority vote among the three closest neighbors.

For bookkeeping, I store per-image correctness in the column vectors log_1nn and log_3nn, where value 1 denotes a correct prediction and 0 an incorrect one. The *hit rate* (accuracy) for 1-NN and 3-NN is the mean of each log vector; multiplied by 100 to express it as a percentage.

```

log_nn = []
log_nn3 = []
repetitions = len(data_train)
for i in range(repetitions):
    nn_1 = Find_3_nn(data_train[i], data_batch_1
, labels_batch_1)
    nn_2 = Find_3_nn(data_train[i], data_batch_2
, labels_batch_2)
    nn_3 = Find_3_nn(data_train[i], data_batch_3
, labels_batch_3)
    nn_4 = Find_3_nn(data_train[i], data_batch_4
, labels_batch_4)
    nn_5 = Find_3_nn(data_train[i], data_batch_5
, labels_batch_5)
    all_nn = nn_1 + nn_2 + nn_3 + nn_4 + nn_5
    all_nn.sort(key=lambda x:x[0])

    if all_nn[1][1] == all_nn[2][1]:
        nnof = all_nn[1][1]
    else:
        nnof = all_nn[0][1]
    log_nn3.append(nnof == labels_train[i])
    log_nn.append(all_nn[0][1] == labels_train[i]
)

nn3stats = np.mean(log_nn3) * 100
nnstats = np.mean(log_nn) * 100

```

Results. For 1-NN the *hit rate* was 35.39% and the testing took 1 hours and 35.45 minutes. For 3-NN the *hit rate* was 35.61% and the testing took 1 hours and 35.45 minutes. The algorithms took the same time since validation was done for both in parallel.

C. Centroid Classifier

The second algorithm that I recreated is the *Centroid Classifier*. The way this algorithm works is by computing the average image-vector (centroid) for each label, and then comparing each testing image to all centroids. The label of the nearest centroid becomes the predicted class.

Implementation of this algorithm is straightforward. First, all training samples are grouped by label, and for each label we build a matrix that contains all image-vectors of that class. The centroid of each class is then computed as the mean vector of its samples.

Listing 4. Computing class centroids

```

all_data = np.vstack([data_batch_1, data_batch_2,
data_batch_3, data_batch_4, data_batch_5])
all_labels = np.concatenate([labels_batch_1,
labels_batch_2, labels_batch_3, labels_batch_4,
labels_batch_5])

# Compute centers for all 10 classes at once (
vectorized)
centers = np.array([all_data[all_labels == c].mean(
axis=0) for c in range(10)])

```

After computing the centroids, the algorithm compares each test image with all ten centroids. The distance used is the Euclidean norm, calculated for all test samples and all classes at once (vectorized). Each test image is then assigned to the class of the closest centroid.

Listing 5. Prediction with Centroid Classifier

```

# Compute distances from each test sample to all
centers at once (vectorized)
# Shape: (num_test_samples, 10)
distances = np.linalg.norm(data_train[:, np.newaxis,
:] - centers[np.newaxis, :, :], axis=2)

# Find nearest center for each test sample
predicted_labels = np.argmin(distances, axis=1)

# Compare predictions with actual labels
log_cc = (predicted_labels == labels_train)

accuracy = np.mean(log_cc) * 100

```

Results. The accuracy (*hit rate*) of the Centroid Classifier is calculated as the mean of the log_cc vector, multiplied by 100 to express it as a percentage. The computation is fully vectorized, resulting in a significant speed-up compared to the nearest neighbor approach.

For this experiment, the Centroid Classifier achieved an accuracy of 27.74%, and the testing time was 3.5 seconds (numpy seems to be pretty optimized).

III. FULLY CONNECTED NEURAL NETWORK FROM SCRATCH

Abstract—For the second part of the assignment, I decided to implement a fully connected (FC) neural network from scratch. After discussing this option with the instructor, I chose it because I believed it would lead to a better understanding of how neural networks operate and learn. Since the main goal of the project was to study how hyperparameters affect learning, I will focus on this aspect rather than on the various implementation “adventures” I encountered while getting the FC network to work.

A. Implementation

My idea was to implement a class representing each layer, encapsulating all the functionality required for it to operate.

Listing 6. Neural Layer Class

```
class FC_layer():
    def __init__(self, Input, Self, batch = 1, rng =
        None, LR = 10**(-4), activation = "relu", W
        = None, b = None):
        self.LR = LR
        self.batch = batch
        self.activation = activation
        self.step = 0
        self.b1 = 0.9
        self.b2 = 0.999
        if isinstance(rng, np.random.Generator):
            rng_gen = rng
        else:
            rng_gen = np.random.default_rng(rng)
        if W is None:
            #W = INPUT x SELF
            if activation == "relu": # good for
                ReLU
                std = np.sqrt(2.0 / Input)
                self.W = rng_gen.normal(0.0, std,
                    size=(Input, Self)).astype(np.
                        float32)
            if activation == "softmax" or activation
                == "sigmoid": # good for sigmoid/
                    tanh if
                limit = np.sqrt(6.0 / max(1, (Input
                    + Self)))
                self.W = rng_gen.uniform(-limit,
                    limit, size=(Input, Self)).
                        astype(np.float32)
            if activation == "nothing":
                self.W = rng_gen.normal(0.0, 0.01,
                    size=(Input, Self)).astype(np.
                        float32)
        if W is not None:
            self.W = W
        if b is None:
            self.b = np.zeros((1, Self), np.float32)
        if b is not None:
            self.b = b
        self.mW = np.zeros_like(self.W)
        self.vW = np.zeros_like(self.W)
        self.mb = np.zeros_like(self.b)
        self.vb = np.zeros_like(self.b)
```

The class stores, as member variables, the number of neurons in the current layer, the number of neurons in the previous layer, the weight and bias arrays, and the first and second moments required by the Adam optimizer, which I chose to use after some research and discussions with other students in the course. Next, we consider the forward-pass implementation.

For a fully connected layer, given a batch of inputs $X \in \mathbb{R}^{B \times d_{in}}$, weights $W \in \mathbb{R}^{d_{in} \times d_{out}}$ and bias $b \in \mathbb{R}^{d_{out}}$, the forward pass is:

$$Z = XW + \mathbf{1}b^\top, \quad (1)$$

$$A = \varphi(Z), \quad (2)$$

where $\varphi(\cdot)$ is the activation function applied elementwise and $\mathbf{1} \in \mathbb{R}^{B \times 1}$ is a column vector of ones (bias broadcasting across the batch).

Listing 7. forward pass

```
def forward(self, IN):
    self.IN = IN
    self.z = IN @ self.W
    self.z += self.b
    self.a = self.func(self.z)
    return self.a
```

Next, we define the activation function method (which is used in the forward pass), which applies the non-linearity specified by `self.activation` elementwise to its input:

Listing 8. function

```
def func(self, x : np.ndarray) -> np.ndarray:
    x = np.asarray(x, np.float32)
    if self.activation == "softmax":
        # keep dtype; or use
        float64 if you like
        # subtract row-wise max for numerical stability
        m = np.max(x, axis=-1, keepdims=True)
        z = x - m
        ez = np.exp(z)
        return ez / np.sum(ez, axis=-1, keepdims
            =True)
    if self.activation == "relu":
        return np.maximum(x, 0.0).astype(np.
            float32)
    if self.activation == "sigmoid":
        x_clip = np.clip(x, -60.0, 60.0)
        return (1.0 / (1.0 + np.exp(-x_clip))).
            astype(np.float32)
    if self.activation == "nothing":
        return x
```

I implemented four activation options: sigmoid, ReLU, softmax, and the identity (“nothing”). However, in the experiments presented in this report, I only used ReLU and softmax.

Next, we present the backward-pass implementation.

For the backward pass, let L be the loss, $X \in \mathbb{R}^{B \times d_{in}}$ the input to the layer, $W \in \mathbb{R}^{d_{in} \times d_{out}}$, $b \in \mathbb{R}^{d_{out}}$, and

$$Z = XW + \mathbf{1}b^\top, \quad A = \varphi(Z),$$

where φ is the activation function and $\mathbf{1} \in \mathbb{R}^{B \times 1}$ is a column of ones.

Given the upstream gradient

$$G_A = \frac{\partial L}{\partial A} \in \mathbb{R}^{B \times d_{out}},$$

the gradient with respect to Z is

$$G_Z = \frac{\partial L}{\partial Z} = \begin{cases} G_A, & \text{for the softmax layer (we assume } G_A = \frac{\partial L}{\partial A} \text{)} \\ G_A \odot \varphi'(Z), & \text{otherwise,} \end{cases}$$

where $\odot$ denotes elementwise multiplication.

Then, with batch size B , the gradients of the parameters and the input are

$$\frac{\partial L}{\partial W} = \frac{1}{B} X^\top G_Z, \quad (3)$$

$$\frac{\partial L}{\partial b} = \frac{1}{B} \sum_{i=1}^B G_Z^{(i)}, \quad (4)$$

$$\frac{\partial L}{\partial X} = G_Z W^\top. \quad (5)$$

For the ReLU activation $\varphi(z) = \max(0, z)$ we have

$$\varphi'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases}$$

Listing 9. backwards pass

```
def backward(self, C_y):
    batch_size = C_y.shape[0]
    if self.activation == "softmax": #dL/dz = y *
        (dL/dy - sum[i] (dLdy(i)*y(i))
        # inner = np.sum(C_y * self.a, axis=-1,
        # keepdims=True)
        # C_z = self.a * (C_y - inner)
        C_z = C_y
    else:
        C_z = C_y * self.dfunc(self.z)
    dW = self.IN.T @ C_z
    db = np.sum(C_z, axis=0, keepdims=True)
    dW /= batch_size
    db /= batch_size

    C_a = C_z @ self.W.T
    #update
    self.step += 1
    b1t = 1.0 - self.b1**self.step
    b2t = 1.0 - self.b2**self.step

    self.mW = self.b1 * self.mW + (1 - self.b1)
    * dW
    self.mb = self.b1 * self.mb + (1 - self.b1)
    * db
    self.vW = self.b2 * self.vW + (1 - self.b2)
    * (dW * dW)
    self.vb = self.b2 * self.vb + (1 - self.b2)
    * (db * db)

    mWhat = self.mW / b1t
    vWhat = self.vW / b2t
    mbhat = self.mb / b1t
    vbhat = self.vb / b2t

    self.W -= self.LR * (mWhat / (np.sqrt(vWhat)
    + 10**(-8)))
    self.b -= self.LR * (mbhat / (np.sqrt(vbhat)
    + 10**(-8)))
    return C_a
```

From the line marked `#update` onwards, we apply the Adam optimizer. For optimization, I used the Adam algorithm. Let g_t denote the gradient at step t (for W or b), and $\beta_1, \beta_2 \in (0, 1)$ the decay rates. Adam maintains first and second moment estimates m_t, v_t :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (6)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (7)$$

where g_t^2 is taken elementwise.

These are then bias-corrected:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad (8)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \quad (9)$$

Finally, each parameter θ_t (e.g. an entry of W or b) is updated as

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \quad (10)$$

where α is the learning rate and ε is a small constant for numerical stability (in my implementation, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\varepsilon = 10^{-8}$).

The method `dfunc` contains the derivatives of the activation functions and is used in the backward pass:

Listing 10. dfunc

```
def dfunc(self, x : np.ndarray) -> np.ndarray:
    x = np.asarray(x, np.float32)

    if self.activation == "relu":
        return (x > 0).astype(np.float32)
    if self.activation == "sigmoid":
        x_clip = np.clip(x, -60.0, 60.0)
        s = 1.0 / (1.0 + np.exp(-x_clip))
        return (s * (1.0 - s)).astype(np.float32)
    if self.activation == "nothing":
        if x.ndim == 0:
            return np.ones((1, 1), dtype=np.float32)
        if x.ndim == 1:
            return np.ones((1, x.shape[0]),
            dtype=np.float32)
    return np.ones_like(x, dtype=np.float32)
```

In summary, this results in a fully functional fully connected neural network layer. It is interesting that such advanced and seemingly high-level models as neural networks can be reduced to relatively simple linear algebra operations.

The full implementation is available in my GitHub repository. There one can also appreciate how much code is required for things such as horizontal flips and grayscale conversions, a custom `crossentropy_withgrad`, computing the loss and tracking it during training, a learning-rate schedule that saves and reloads weights from files (with an entirely scientific 5-second wait to make sure they are really there), wiring the layers together, computing training and testing accuracy, keeping the correct labels aligned with each mini-batch, and implementing the `train` and `test` functions while shuffling batches without losing their labels. The repository also includes an unsuccessful attempt at implementing dropout, which I decided to keep as a reminder that not every experiment works on the first try. In addition, I stored the weights and biases for all networks used to generate the learning curves and plots in the next section, so that anyone sufficiently motivated can reload them and reproduce the experiments exactly.

IV. EXPERIMENTS AND RESULTS ON FC

In this section I present the experiments I ran with the fully connected network, focusing on how different hyperparameters affect training and generalization. For each experiment I show the corresponding learning curves and briefly discuss the observed behaviour.

All experiments were run on the CIFAR-10 dataset using the fully connected architecture described in the previous section. Unless otherwise stated, I used ReLU activations in the hidden layers, a softmax output layer, and cross-entropy loss.

A. Zero Hidden Layers

The first plot I generated was the one for the FCN with 0 hidden layers and learning rate (LR) at 0.001.

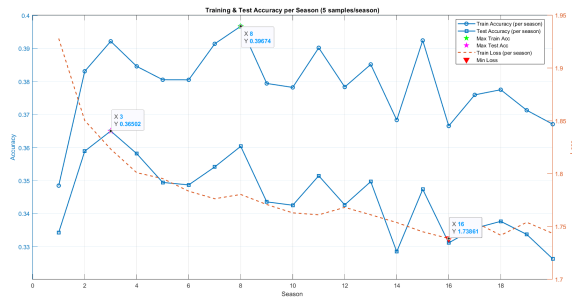


Fig. 1. Training and test accuracy and training loss per season.

From this graph I understood that CIFAR-10 is a hard problem to solve and that 0 hidden layers just won't cut it, since we see a classic underfitting problem. Before changing the architecture, I wanted to see 1) how adding flipped training data and 2) how making the pictures black and white would change the training graph and the performance of the model.

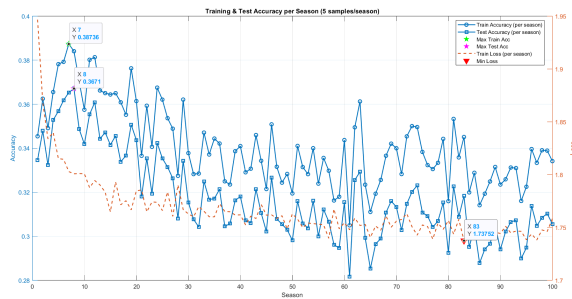


Fig. 2. Training and test accuracy and training loss per season.

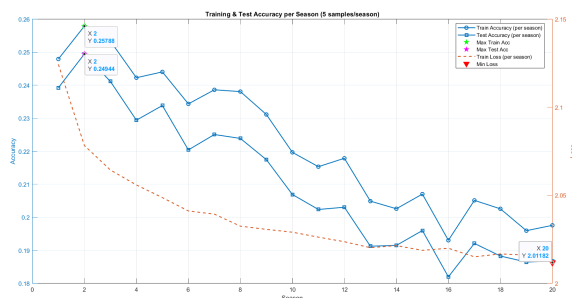


Fig. 3. Training and test accuracy and training loss per season.

My understanding of these graphs is that, since the network is so weak, it just would not be able to perform "nice". "Nice"

here (for me) means doing better than KNN, so achieving a test accuracy higher than 35.61%.

Also, I had a heads up that black and white images, despite making the input shorter, actually make the problem more difficult, so I was not expecting a magical boost in performance from that change alone.

B. One Hidden Layer

The first training I tried was with 1 hidden layer of 256 neurons, and here is the graph:

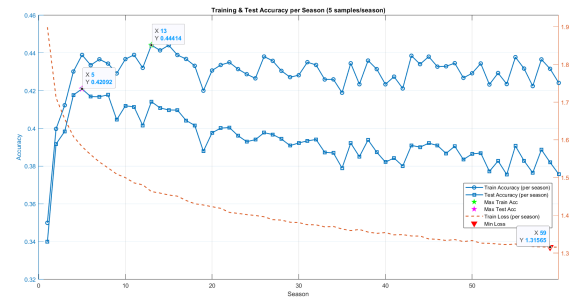


Fig. 4. Training and test accuracy and training loss per season.

Hurray! Test accuracy at 42%, so we are officially out of the KNN territory, meaning our network is doing something smarter than just counting distances between images. The LR is still at 0.001, and the thing that bothered me the most is that after season 10 the performance didn't really improve, which basically means my laptop was running for no real reason after that.

The next model I implemented was the same one, just with black and white images:

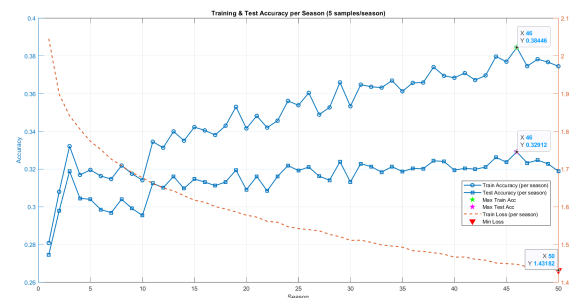


Fig. 5. Training and test accuracy and training loss per season.

Since there was this really gradual ascent, I figured that the experiment had potential, so I doubled the size of the hidden layer and ran a black and white version with 512 neurons:

Here things again took a fatal path for black and white, because I had accidentally left `dropout = true` (on my side I didn't even know yet that my dropout implementation didn't actually work). I was very disappointed and didn't continue running black and white experiments, which is something I

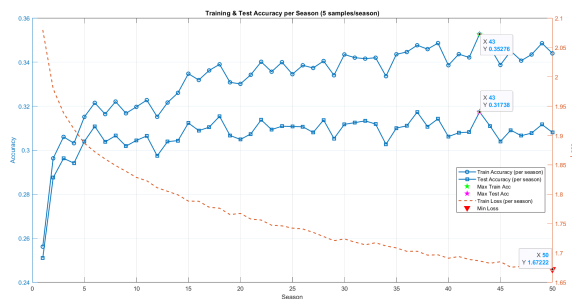


Fig. 6. Training and test accuracy and training loss per season.

now regret, but at the time the deadline was coming up and I just didn't have the time.

The thing I learned though, from observing these graphs compared to the zero hidden layer networks, is that when the line is relatively smooth it usually means the network is actually doing something and heading somewhere. When it looks more like the graphs from the zero hidden layer case, it basically means the network is really weak and can't properly train on the given samples.

C. Two Hidden Layers

The first training I tried with two hidden layers used 500 and 100 neurons:

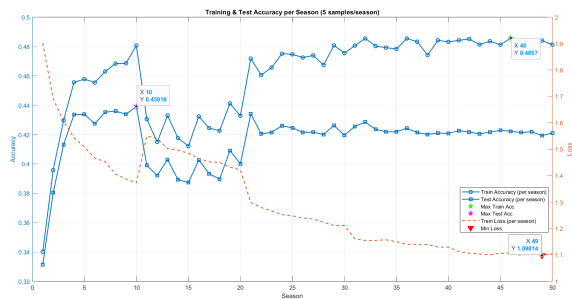


Fig. 7. Training and test accuracy and training loss per season.

The results were not really improving, and that dent in the graph is something that I couldn't wrap my head around. So the next model I made had 2000 and 1000 neurons, because I thought there was a size issue again (later I realized that a potential issue was also that I went from 100 to 10 neurons in the last hidden layer, which was probably too small).

(Excuse the style of this graph, it comes from my old MATLAB plotting script, and at that point I had already lost the loss / training / testing arrays, so I couldn't recreate the nicer graphs from above.)

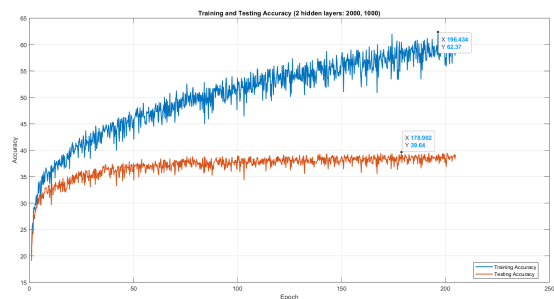


Fig. 8. Training and test accuracy and training loss per season.

The results here were even worse, and I started realizing that 256 neurons for the last hidden layer seemed to be the sweet spot.

D. Three Hidden Layers

The first training I tried with three hidden layers used 1024, 500 and 100 neurons (so here I was basically cancelling what I said above about 256 being the sweet spot, because I wasn't sure yet and was ready to run the same setup with 256 neurons in the last hidden layer to confirm):

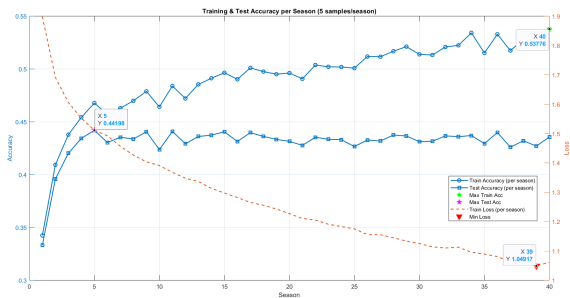


Fig. 9. Training and test accuracy and training loss per season.

I manually ended this training because I saw the testing accuracy not improving.

Then I tried the version with 256 neurons in the last hidden layer:

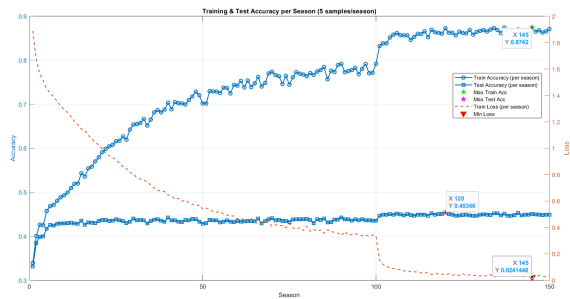


Fig. 10. Training and test accuracy and training loss per season.

This training graph was something that I really stared at and was super proud of. I finally had a new problem: overfitting. My network was now really strong and was mostly improving only the training accuracy. For this experiment I also used a learning rate schedule: the LR was 0.001 up to season 50, then 0.0004 up to season 100, and finally 0.0001 from season 100 to season 150. To fix this, I implemented dropout, flipped all training images, and ran an even bigger network with both applied.

E. Five Hidden Layers

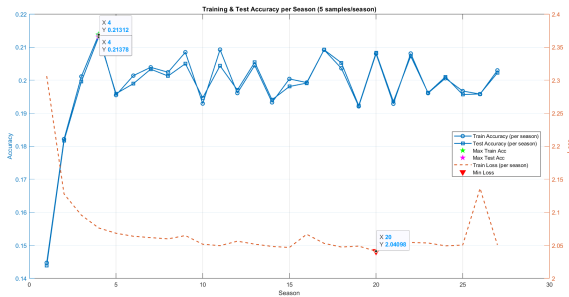


Fig. 11. Training and test accuracy and training loss per season.

Here I realized that either the flip augmentation or dropout (or both) did not work as intended, and with the deadline coming up I bet that dropout was the issue and decided to run what I thought would be my last training experiment with four hidden layers.

F. 4 Hidden Layers

2048 1024 512 256 neurons and flips

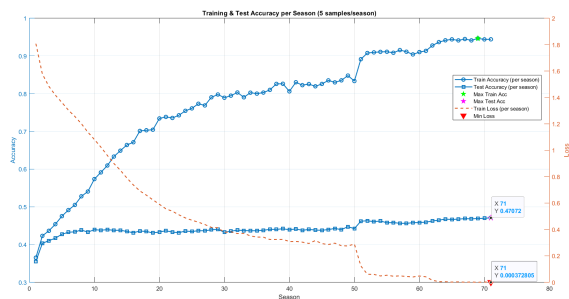


Fig. 12. Training and test accuracy and training loss per season.

So the all-time high testing accuracy I got was 47%, which to me actually feels quite impressive for a from-scratch fully connected network on CIFAR-10. Another training run I managed to do was a deeper 6-layer network with 100 neurons in each layer. I think this model also had some potential; I don't necessarily think it would beat the current all-time high, but with more time I would have liked to try more “many layers with fewer neurons” architectures and see how far they could go.

V. FINAL THOUGHTS

Overall, I really enjoyed working on this project. It was the biggest project I have done so far, and I learned a lot—not only about neural networks, but also about programming, debugging, and organizing my time around long experiments.

If I had more time, one of the main things I would like to fix is my dropout implementation, because I am pretty sure that a correctly implemented dropout layer would help with the overfitting I observed in the deeper models. I would also have liked to implement a CNN and run more experiments to compare it fairly against my fully connected networks. However, the from-scratch implementation already took me a lot of time, mostly because of various bugs and small, sometimes quite dumb issues that I had to track down and fix.

A. Oh and one more thing...

Since I had saved all the weights for my strongest network, I decided to try visualizing them. The result turned out, in my opinion, pretty impressive and gives a nice picture of the overall connectivity structure of the network.

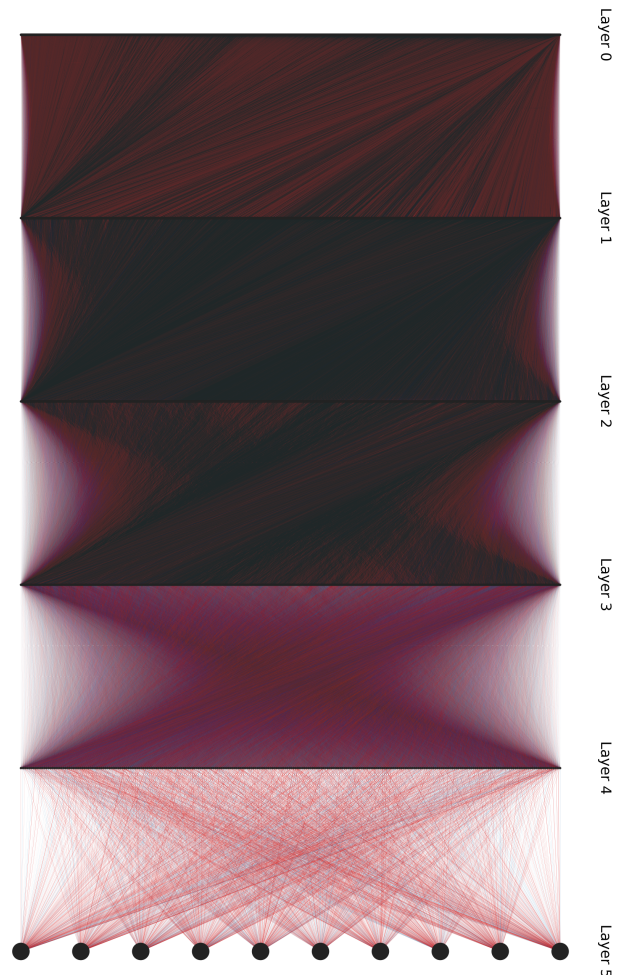


Fig. 13. Neural Network Visualization.